Assignment 5: Build, Measure, and Explain Scheduling Classes, Priority, and Delay Attribution

Linux Systems and Performance - Week 5

Deadline Frame Loop -> Priority-Propagating Service -> Wake-up Delay Attribution

1. Assignment Philosophy

This assignment is not evaluated only by whether the code runs. You are given a frame loop that runs inside a SCHED_DEADLINE reservation and a service that passes its caller's priority on to the thread doing the work. You write the bpftrace tools that check from kernel events whether they behave as intended, plus a tool that explains why a thread waited, and you explain the results with the Week 5 scheduling model.

Each level pairs a provided program with a bpftrace tool you write. When the numbers the program measured itself and the kernel-side numbers from the same run disagree, explain why. That disagreement is the most important thing to observe.

PREDICT -> BUILD -> MEASURE -> ANALYZE -> EXPLAIN

Important: AI coding tools may be used. However, you are responsible for understanding, debugging, modifying, and explaining all submitted code and experimental results. Implementation alone is not sufficient for full credit.

2. Challenge Levels

Level	Build	Load / Trigger	Measure / Observe	Main Question
Level 1	frametrace.bt for the provided deadline frame loop (frameloop)	spike frames, overrun mode, background load, policy	miss, periods used, per-frame on-CPU and throttled time, SIGXCPU timing	What does CBS do when a frame overruns its runtime, and who pays for it?
Level 2	reqtrace.bt for the provided priority-propagating service (svc)	RT hog, client priority, queue order	per-segment request latency, worker kernel prio, send_late	How does a caller's priority reach the thread that does its work?
Level 3	delay attribution tool (whydelay.bt)	mystery cases 1 to 4, the Level 2 svc	on-CPU vs. waiting time, blamed tasks and their class	Is a late frame a program problem or a policy problem?

Levels 1 and 2 are independent, so you may start with either. Level 3 is applied to the provided mystery program and to the Level 2 service. Absolute performance across different machines is not graded. Kernel modification is not required.

3. Common Interface and Rules

- Write the tracing tools in bpftrace unless otherwise approved. Run on your laptop with WSL2 kernel 6.6 and the bpftrace installed in Lab 5 step 0. Use two vCPUs (processors=2) unless otherwise specified. The provided programs run their RT threads on CPU 0. DEADLINE threads are the exception because they cannot be pinned to a CPU.
- Build with the provided Makefile (-g -fno-omit-frame-pointer). Keep machine, build options, input, and competing load fixed.
- Fixed parameters: frameloop uses period 16667 us, runtime 4000 us, 2500 us for a normal frame, 6000 us for every 10th frame (id % 10 == 9), and a budget of 3500 us. The svcload clients and hog (priority, period, work, start time) and the mystery cases are fixed as well. Everyone uses the same values, so results are comparable across reports.
- The programs in src/ are provided complete so that every report measures the same behavior. Do not modify them. You may add your own analysis scripts. The uprobes attach to frame_begin, frame_end, req_enqueue, handle_begin, handle_end, req_send, and req_done. Run bpftrace from inside the assignment folder, because the scripts find the programs through ./bin/... paths.
- Predict before running each experiment. Repeat measurements. Record exact commands, PID, uname -r, and execution time in raw logs saved with script -c or tee. Every measurement row must identify its source log. Keep bpftrace output as the original file.
- After every experiment, check `sudo dmesg | grep -i throttl` and mark any run affected by RT throttling. When the program and bpftrace disagree, keep both numbers and explain the difference.
- Document kernel-version-dependent details when they affect the implementation or observed behavior. Screenshots alone are not sufficient. Include short interpretations and cite your own measurements.

4. Required Interface and Test Tools

```
sudo ./bin/frameloop [-p dl|fifo:P|other] [-g none|signal|budget] [-n frames]
                     [-B budget_us] [-o out.csv]
sudo ./bin/svc [-i] [-q fifo|prio] [-b base_prio] [-R rx_prio] [-d seconds]
./bin/svcload [-d seconds] [-H hog_prio] [-o out.csv]
sudo bpftrace bt/frametrace.bt -c './bin/frameloop ...'
sudo bpftrace bt/reqtrace.bt <seconds>
sudo bpftrace bt/whydelay.bt <PID> <thread comm> <seconds>
sudo ./scenarios/mystery <case 1..4> [seconds]
bash tools/l2_run.sh <tag> [svc options]
bash tools/l3_run.sh case <1..4> | fix1 | svc <tag> [svc options]
bash tools/burn.sh [N] [seconds]
```

In the skeleton (Assignment5_skeleton_code.zip), frameloop, svc, and svcload are complete. The three bt scripts run but leave their maps empty until you fill in the TODOs. l2_run.sh and l3_run.sh ask for the sudo password once, so type one command at a time. Use chrt -p, ps -L -o tid,psr,cls,rtprio,comm, and dmesg to validate runtime behavior.

File	What You Do
bt/frametrace.bt	TODO(L1-5, L1-6) on-CPU and throttled time, SIGXCPU records
bt/reqtrace.bt	TODO(L2-5) queue / service / reply segments and worker maps (rx is given as an example)
bt/whydelay.bt	TODO(L3-1) the whole script (design and hints in the comments)
src/frameloop.c, src/svc.c	provided: read the lines marked KEY, which the explanations ask you to cite. Do not modify

5. Level 1 - Deadline Frame Loop

5.1 Provided Program and Required Implementation

- frameloop (provided) processes one frame per period. Each frame has a fixed amount of work (2500 us normally, 6000 us for a spike), measured in thread CPU time and processed in 100 us chunks, so a chunk boundary is the point where a frame can be stopped.
- With -p dl it sets SCHED_DEADLINE with sched_setattr (runtime 4000 us, deadline = period = 16667 us) and SCHED_FLAG_DL_OVERRUN, so the kernel sends SIGXCPU when the runtime is exceeded. The SIGXCPU handler only sets a flag and counts. After a frame it gives back the rest of the runtime with sched_yield().
- Overrun modes: -g none always finishes the frame. -g signal stops the frame at the first chunk boundary after SIGXCPU arrives and records DROP. -g budget checks before each chunk whether "CPU time used in this frame + the next chunk" would exceed the budget (3500 us), and if so stops and records CUT. Find the KEY lines for the reservation, the overrun flag, and the DROP and CUT decisions in frameloop.c.
- What you implement, frametrace.bt (TODO L1-5, L1-6): print one line per frame, `F <id> <wall_us> <oncpu_us> <offcpu_runnable_us> <n_out> <sigxcpu> <sig_at_us> <status>`. oncpu_us is summed from sched_switch. offcpu_runnable_us and n_out are the time and count of being off the CPU while still runnable, which under DEADLINE is throttling. sigxcpu and sig_at_us come from tracepoint signal:signal_generate (count, and time of the first one from frame start).
- Validation: join the program CSV and the frametrace output of the same run by frame id (any scripting language). Count frames where wall_us or oncpu_us differ by more than 200 us, and find what those frames have in common.

5.2 Required Experiments

```
for g in none signal budget; do
  sudo ./bin/frameloop -g $g -n 600 -o logs/l1-$g.csv
done

for g in none signal budget; do
  sudo bpftrace bt/frametrace.bt -c "./bin/frameloop -g $g -n 300 -o logs/l1-$g-bt.csv" \
    > logs/l1-$g-bt.txt
done

bash tools/burn.sh $(nproc) 120 &
for p in dl fifo:50 other; do
  sudo ./bin/frameloop -p $p -g none -n 600 -o logs/l1-burn-${p%%:*}.csv
done
```

Report a compact comparison table. Add the exact configuration and source-log path to each row.

Comparison	Prediction	miss / periods_used / work_done	oncpu / offcpu / sig_at	Interpretation
none / signal / budget				
program vs. frametrace, with / without bpftrace				
dl / fifo:50 / other under burn.sh				

5.3 Required Explanation

1. Compare miss, periods_used, and work_done for none, signal, and budget. In none, show with oncpu_us and offcpu_runnable_us how many periods one spike frame uses, and explain it with how CBS treats a task that has used up its runtime (throttle and replenish).

2. Did signal mode reduce misses? Use sig_at_us to show when the kernel generated SIGXCPU. If frames that were not spikes were dropped, explain why. Can SIGXCPU be used to detect an overrun, or to prevent one? Answer with evidence, citing the KEY lines of the DROP and CUT decisions.

3. Show the distribution of the difference between frametrace oncpu_us and the program's cpu_us (CLOCK_THREAD_CPUTIME_ID). What should you suspect for frames with a large difference? Measure the observation cost as the change in frame_wall p50 with and without bpftrace, and explain where it comes from.

4. Under background load, compare miss and frame_wall p99 for dl, fifo:50, and other. How did FIFO handle the spike frames, and who paid for it? Why are non-spike frames late under other? For a webOS frame pacing thread, which policy would you choose, and on what basis would you set its runtime?

6. Level 2 - Priority-Propagating Service

6.1 Provided Program and Required Implementation

- svc (provided) receives requests over an abstract unix socket (SOCK_SEQPACKET). Both of its threads run as SCHED_FIFO on CPU 0. svc-rx (priority 50) only reads requests, queues them, and wakes the worker. svc-worker (priority 10) takes requests one at a time, burns work_us of CPU, and replies.
- Priority handling: set_worker_prio() changes the worker's SCHED_FIFO priority with pthread_setschedparam and counts boosts. -q prio takes the highest client priority first, arrival order among equals. With -i the receiver raises the worker to a request's priority before waking it, and the worker serves each request at max(base, that request's priority, highest priority still queued), returning to base when the queue is empty. The queue lock is a priority inheritance mutex. Find these KEY lines in svc.c.
- What you implement, reqtrace.bt (TODO L2-5): join req_send, req_enqueue, handle_begin, handle_end, and req_done by request id, and split each request into rx, queue, service, and reply segments with a per-client histogram for each. Also print @svc_wait (time the worker was off the CPU while runnable during processing), @preempted (who took the CPU), @prio_at_begin (worker kernel prio just before handle_begin), and @reported_prio. Run it for a fixed time instead of using -c, because uprobes go into both svc and svcload.
- svcload (do not modify): cl-input (FIFO 30) sends one 2 ms request every 50 ms and waits for the reply. cl-logger (FIFO 15) sends six 1 ms requests every 20 ms. hog (FIFO 20) burns 30 ms of CPU every 100 ms and never talks to svc. Half of the input requests arrive right after the hog starts. svcload prints per-client response time and send_late (how late each request was sent).

6.2 Required Experiments

```
bash tools/l2_run.sh none
bash tools/l2_run.sh prio    -q prio
bash tools/l2_run.sh inh     -i
bash tools/l2_run.sh inhprio -i -q prio
HOG=40 bash tools/l2_run.sh hog40 -i -q prio
HOG=0  bash tools/l2_run.sh nohog
```

l2_run.sh starts svc, attaches reqtrace.bt, and runs svcload for 10 seconds. Report a compact comparison table. Add the exact configuration and source-log path to each row.

Configuration	Prediction	input p50 / p99, send_late	queue / service / svc_wait	@prio_at_begin / served_prio	Interpretation
none / prio					
inh / inhprio					
hog40 / nohog					

6.3 Required Explanation

5. Quote input response p50 and p99 for none and inhprio. Use reqtrace to show which segment shrank, and use @preempted to name the task that preempted the worker without priority inheritance. Check whether @prio_at_begin agrees with @reported_prio.

6. Compare inh (FIFO queue) with inhprio. From input p99 and the served_prio distribution of the logger, explain why some logger requests are processed at priority 30 when priority inheritance is on with a FIFO queue. What does that cost the input client?

7. svc boosts the worker in the receiver, before waking it, instead of letting the worker boost itself after taking a request (cite the KEY line). Use the queue segment of the none run to explain why. When does the kernel's PI futex boost the owner (piwatch.bt in Lab 5 step 2)? Why can the kernel not do it for this service?

8. In hog40 the input response time is close to inhprio. What got worse? Explain with send_late, say what a response-time-only measurement misses, why priority inheritance does not fix it, and how you would fix it.

9. svc uses a priority inheritance mutex for the queue lock (cite the KEY line). If it were a plain mutex, what could happen to svc-rx? Describe the scenario with the priorities of svc-rx, svc-worker, and hog (the experiment is optional).

7. Level 3 - Wake-up Delay Attribution

7.1 Required Implementation

- whydelay.bt measures the time a thread with the given name in the target process was runnable but did not get a CPU. A wait starts when the thread wakes up (sched_wakeup) or leaves the CPU while still runnable (preempted or throttled), and ends when it gets the CPU again.
- Output (TODO L3-1): @oncpu_us (time the target worked on a CPU), @wait_us, @wait_hist_us, and @wait_max_us (total, distribution, and maximum of its waits), @victim_prio (kernel prio of the target when it gets the CPU, which shows its own class), and @blame_us[comm, prio] (for each task that ran on that CPU while the target waited, the run time that overlapped the wait). Waits are accounted on the CPU where they started.
- sched_wakeup can fire for a task that is already running. If you do not filter those events, waiting time is inflated. See the hints in the skeleton comments.
- mystery starts a ui-render thread (one frame every 16.7 ms) and a few other threads, and prints only ui-render's frame statistics. It does not tell you why each case is slow, and its source is not provided. Find the cause with whydelay.

7.2 Required Experiments

```
for c in 1 2 3 4; do bash tools/l3_run.sh case $c; done
bash tools/l3_run.sh fix1
bash tools/l3_run.sh svc noinh
bash tools/l3_run.sh svc inhprio -i -q prio
```

In case mode l3_run.sh measures ui-render for 8 seconds. fix1 switches the thumbgen threads of case 1 to SCHED_IDLE before measuring. svc mode measures svc-worker of the Level 2 svc for 12 seconds. Report a compact comparison table. Add the exact configuration and source-log path to each row.

Case	Prediction	frame_time_p99 / miss	@oncpu_us / @wait_us	Blamed Tasks (comm, prio)	Verdict and Fix
1					
2					
3					
4					
fix1					
svc-worker (no -i / -i -q prio)					

7.3 Required Analysis

10. Tabulate the four cases: symptom (frame_time_p99, miss), @oncpu_us and @wait_us, the tasks that caused the wait and their class, the verdict (program problem or policy problem), and the fix. Quote the map values behind each verdict.

11. In case 2, compare the prio of the tasks in @blame_us with ui-render's @victim_prio. Explain why those tasks cannot preempt ui-render, what the real cause is, and which Level 1 observation this verdict connects to.

12. In case 1, compare whydelay results and frame statistics before and after switching thumbgen to SCHED_IDLE. Give two other fixes with the same effect and the cost of each.

13. Compare the two svc-worker results. Input response time dropped sharply, but what happened to svc-worker's overall @wait_us and to the hog's @blame_us? Explain why, and what this shows about a limit of whydelay. Name at least one other limit (accounting on the CPU where a wait started versus migration, the gap between the sum of @blame_us and @wait_us, observation cost, and so on), and quote the number in your own measurements where that limit shows.

8. Evidence Required

Level	Implementation Evidence	Runtime / Measurement Evidence	Analysis Evidence
1	frametrace.bt (on-CPU, throttled time, SIGXCPU) + program/frametrace join by frame id	none/signal/budget comparison, with / without bpftrace, policies under burn.sh	CBS throttle and replenish, overrun detection, observation cost
2	reqtrace.bt (four segments, @svc_wait, @preempted, @prio_at_begin)	six l2_run conditions, segment histograms, @prio_at_begin vs. @reported_prio, send_late	priority inversion, PI futex vs. IPC, head-of-line blocking
3	whydelay.bt with the running-task wakeup filter	four mystery cases + fix1, svc-worker with and without -i	program vs. policy problem, throttling vs. preemption, tool limits

9. Submission Package

- bt/frametrace.bt, bt/reqtrace.bt, bt/whydelay.bt, any analysis scripts you wrote, and the original log directory.
- README with kernel version, vCPU count, bpftrace version, exact build/run commands, and highest completed level.
- Technical report (recommended 4-6 pages) organized as Level 1 / Level 2 / Level 3. Include predictions, controlled comparisons, measured results, and explanations.

- Level 1 overrun-mode and policy tables with the program/frametrace join, Level 2 request-segment tables, and the Level 3 case table.
- Selected runtime evidence and exact commands. Every measurement row must identify its raw log. Do not submit pages of unannotated raw output.
- A short AI-use statement if AI tools were used, including at least one generated suggestion or code issue that you verified, changed, or rejected.

10. Evaluation

Category	Weight	What Is Evaluated
Implementation	25%	three bpftrace tools whose output matches the spec and lines up with the program records of the same run, including the running-task wakeup filter in whydelay
Experiment & Measurement	35%	predictions, fixed parameters, PID/kernel logs, original bpftrace output, RT throttling marked, and repeatability
Technical Explanation	30%	reasoning from measured values and the provided code about CBS and throttling, PI futex vs. IPC, gaps between program and kernel measurements, and program vs. policy problems
Code Quality & Report	10%	check-off modifications, reproducibility, and concise evidence

Performance is not graded by absolute numbers across different hardware. The quality of the controlled comparison, the correctness of the measurement method, and the explanation of observed behavior are graded.

A program that appears to work but is not experimentally validated or correctly explained cannot receive full credit.

11. Final Check-Off Questions

The oral check-off takes approximately five minutes. Predict each result before making the change and running it.

- Change the frameloop budget to 3900 us (-B 3900). Predict miss and the number of throttled frames in frametrace before running.
- Lower the svc receiver priority to 15, below the hog (-R 15). Before running with -i -q prio, predict input p99 and which reqtrace segment will grow.
- Remove the condition in whydelay.bt that filters wakeups of already running tasks. Predict how @wait_us changes in case 4, then explain the result.
- In mystery case 3, raise ui-render to SCHED_FIFO 60 (chrt -f -p 60). Predict how the waits of ui-render and audio-mix change before showing the measurement.

Build it. Measure what changes. Explain why.